Python: возврат нескольких значений: 4 метода и примеры

Кимберли Фессел

Языки программирования дают нам удивительную возможность выполнять повторяющиеся задачи с небольшими изменениями, используя пользовательские функции. Вы можете создавать такие функции в Python и настраивать их так, чтобы они возвращали ноль, одно, два или более значений.

Оглавление

В этой статье мы покажем вам несколько способов получения нескольких значений из функции Python. Мы сравним преимущества каждого из них и предупредим вас о нескольких распространенных ошибках, которых следует избегать. Начнем с краткой демонстрации.

Возврат нескольких значений из функции

Определить функцию в Python так же просто, как ввести ключевое слово `def` и дать функции имя. Вы пишете код с инструкциями о том, что должна делать ваша функция, и при желании можете сделать так, чтобы функция возвращала информацию с помощью оператора `return`. Вот пример кода функции, которая принимает два аргумента и при вызове возвращает их сумму:

python

```python
def add_two(x, y):
    total = x + y
    return total

print(add_two(2, 3))

# Expected result:
# 5
```

Если вы хотите, чтобы функция возвращала несколько значений, разделите их запятыми в операторе `return`. Например, эта функция вычисляет сумму и произведение входных значений:

python

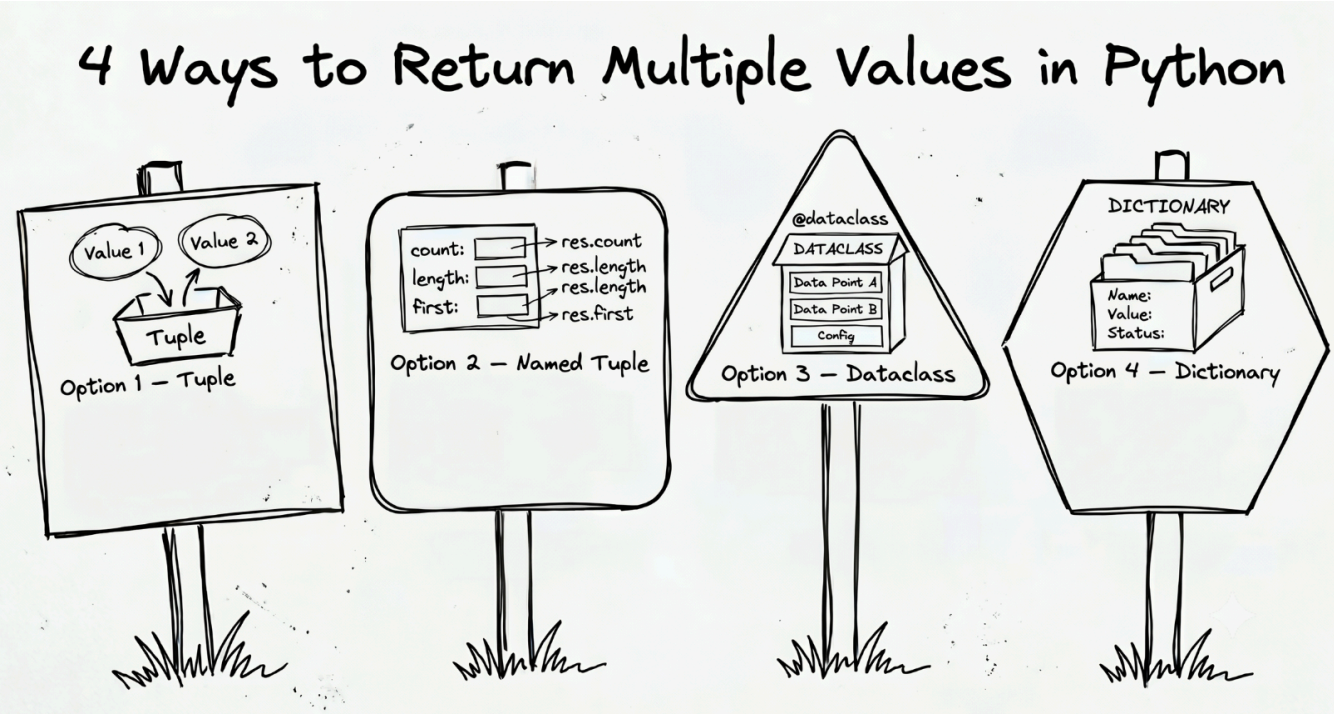
```python
def add_multiply_two(x, y):
    total = x + y
    product = x * y
    return total, product

print(add_multiply_two(2, 3))

# Expected result:
# (5, 6)
```

Python автоматически упаковывает возвращаемые значения в **кортеж**. Этот кортеж содержит оба выходных значения в том же порядке, в котором они перечислены в `return`.

The tuple is the default and generally most Pythonic way to return more than one value, but there are a few other solutions worth mentioning as well.



Option 1 — Tuple

Python functions return exactly one object, so to get more than one value, you'll need to package them up in a larger collection. More often than not, you'll see multiple values returned as a tuple. The tuple data type is immutable, and it preserves the order of elements as defined at creation, which turns out to be perfect for packaging related values together.

Use the return Statement for Multiple Return Values

You can put multiple values, separated by a comma, in `return`. Python implicitly creates a tuple containing all your return items for you, so you can, but do not need to, enclose those values in parentheses.

Here's a function called `analyze_text()` that outputs the length, word count, and an uppercase version of an input text string:

python

```python
def analyze_text(text):
    length = len(text)
    word_count = len(text.split())
    upper = text.upper()
    return length, word_count, upper

print(analyze_text("Now returning multiple values."))

# Expected result:
# (30, 4, 'NOW RETURNING MULTIPLE VALUES.')
```

Unpacking Multiple Variables from the Returned Tuple

Because Python automatically packages your values up in a tuple, you can either access the individual values by index position or unpack them to have direct access. Here's each option in code:

1. Save the resulting tuple as a single variable and reference the individual items by index position.

```python
# Option 1 - Save as a single item
summary = analyze_text("Now returning multiple values.")

print(f"There are {summary[1]} words in the string.")

# Expected result:
# There are 4 words in the string
```

1. Save each value individually for immediate access.

```python
# Option 2 - Save individuals
length, word_count, upper = analyze_text("Now returning multiple values.")

print(f"There are {word_count} words in the string.")

# Expected result:
# There are 4 words in the string
```

This option works well for simple, fixed-structure returns. Keep in mind, however, the number of comma-separated variables on the left must match the number of returned values, or Python will raise a `ValueError`. Also note the significance of order here. The returned tuple contains no labels. You'll need to remember the first value contains the length and so on to avoid errors.

Option 2 — Named Tuple

For improved readability, consider sending back several values in a **named tuple**. The `namedtuple` data type comes from the built-in `collections`

module. It's immutable and retains order like regular tuples, but it also allows you to look up elements by name with dot notation.

You can store multiple values in a named tuple and have your function output it as a single Python object. Here's code to once again analyze a text string, but with the results now stored in a named tuple:

python

```python
from collections import namedtuple

Analysis = namedtuple("Analysis", ["length", "word_count", "upper"])

def analyze_text(text):
    return Analysis(
        len(text),
        len(text.split()),
        text.upper()
    )

summary = analyze_text("Now returning multiple values.")

print(summary)
print(summary.length)
print(summary.word_count)

# Expected result:
# Analysis(length=30, word_count=4, upper='NOW RETURNING MULTIPLE VALUES.')
# 30
# 4
```

Named tuples are good for fixed-structure situations where clarity matters. You no longer need to remember the exact position of each element because you can refer to it by name. They are a nice middle ground between plain tuples (fast but positional) and full classes (flexible but more complex).

Option 3 — Dataclass for Complex Return Values

Moving beyond tuples, **dataclasses** allow greater flexibility for multiple return values. They come from the `dataclasses` module, and you can define them with a simple `@dataclass` decorator. They're designed for structured objects but offer more capability than named tuples.

Here's what `analyze_text()` looks like with a dataclass:

python

```python
from dataclasses import dataclass

@dataclass
class Analysis:
    length: int
    word_count: int
    upper: str

def analyze_text(text):
    return Analysis(
        length=len(text),
        word_count=len(text.split()),
        upper=text.upper()
    )

summary = analyze_text("Now returning multiple values.")

print(summary)
print(summary.length)
print(summary.word_count)

# Expected result:
# Analysis(length=30, word_count=4, upper='NOW RETURNING MULTIPLE VALUES.')
# 30
# 4
```

Unlike tuples, dataclass objects are mutable by default, meaning you can modify their values after creation.

python

```python
summary.word_count = 10
print(summary)
```

```
# Expected result:
# Analysis(length=30, word_count=10, upper='NOW RETURNING MULTIPLE VALUES.')
```

You can also include default values and even add methods for more advanced behavior.

Option 4 — Dictionary

Dictionaries give you yet another solution for returning several results from a single function call. They store information in key-value pairs, so you can look up data by descriptive keys instead of position. Dictionaries are **mutable**, so you can easily add, remove, or change fields on the fly.

For example, the `analyze_text()` function could return a dictionary of information with the computed values when you call it:

python

```python
def analyze_text(text):
    return {
        "length": len(text),
        "word_count": len(text.split()),
        "upper": text.upper()
    }

summary = analyze_text("Now returning multiple values.")

print(summary)
print(summary["length"])
print(summary["word_count"])

# Expected result:
# {'length': 30, 'word_count': 4, 'upper': 'NOW RETURNING MULTIPLE VALUES.'}
# 30
# 4
```

Give dictionaries a try if your return structure is not fixed or may evolve over time. They're commonly used in data processing tasks and APIs as they map naturally to JSON-like output.

The Quick-Reference Comparison Table

Here's a handy table for you to compare and contrast the techniques we covered for returning multiple values in Python:

Technique	Mutable/Immutable	Element Reference	Readabili
Tuple	Immutable	Position	Medium
Named Tuple	Immutable	By name (dot notation) or position	High
Dataclass	Mutable (by default)	By name (dot notation)	High
Dictionary	Mutable	By key (square brackets)	High

AI Tutor: Show me Python examples of getting more than one value from a function with lists and generators.

Common Errors and Best Practices for Python Functions

With the multiple ways to package results, there's bound to be a few things to watch out for in your code. Keep these tips in mind to avoid errors and make the most of your data structures.

Mismatched Unpacking of Return Values

Perhaps the most common error regarding this topic is having too few or too many comma-separated variables when attempting to unpackage your returned results. For example, if you write:

```
a, b = func()
```

Python expects exactly two return values for the two variables, `a` and `b`. You'll get an error for any more or less than expected.

When the function actually gives you more values than you unpacked, you'll get a `ValueError`:

```python
def func():
    return 1, 2, 3

a, b = func()

# Expected result:
# ValueError: too many values to unpack (expected 2)
```

Likewise, you'll also get an error if the functions returns too few outputs:

python

```python
def func():
    return (1,)

a, b = func()

# Expected result:
# ValueError: not enough values to unpack (expected 2, got 1)
```

If you receive a `ValueError`, double-check that the number of variables is equal to the number of returned values.

Using _ to Skip Outputs

You'll need to assign the exact number of outputs a function provides if you choose to extract the values, but if you don't need a particular output, you can assign it to the throwaway variable `_`. Developers understand a single underscore as a placeholder for a value you don't care about.

To test this out, we can assign the uppercase version of our text from `analyze_text()` to `_` if we don't need it:

python

```python
def analyze_text(text):
    length = len(text)
    word_count = len(text.split())
    upper = text.upper()
    return length, word_count, upper

length, word_count, _ = analyze_text("You don't need this uppercase.")

print(word_count)
print(_)

# Expected result:
# 5
# "YOU DON'T NEED THIS UPPERCASE."
```

Choosing the Simplest Structure that Works

Even with the comparison chart, you may still be wondering when to choose each of the options discussed in this article. Ultimately, you'll want to pick the simplest technique that supports your project. Start with the simple tuple and increase the complexity as needed. Specifically:

- Use a **tuple** for simple, fixed outputs
- Use a **named tuple** when readability is important
- Use a **dataclass** for structured data when you expect to append fields or methods later
- Use a **dictionary** when fields are optional or dynamic

Wrapping Up

When it's time to return multiple values, Python allows several different techniques. Remember that functions return exactly one object, so you'll need to package your values into a collection of some sort. Separate your values with commas in the return statement, and Python will automatically wrap them into a tuple. If you need additional readability or flexibility, consider named tuples or dataclasses, and move on to dictionaries when fields are optional or may change over time.

No matter which way you decide to package up your return values, you can learn more by interacting with the AI Tutor. This prompt should get you going:

AI Tutor

Give me a few real-world Python examples of returning multiple values from functions.

Related Guides

Python KeyError Exceptions: Causes and Fixes Explained

Python Null (None): Guide to Missing Values and NoneType

Python Backend Development: Build Your First API

Python Backend Frameworks: How to Choose the Right One

Python Multiline Strings: The Complete Guide

Python not Operator: The Complete Guide to Logical Negation

Python Print New Line: Methods, Examples, and Best Practices

Python Exponent: 5 Methods for Exponentiation + Applications

Python Modulo Operator (%): Complete Guide with Examples

Python Division: Operators, Floor Division, and Examples

Join the Community

roadmap.sh is the 6th most starred project on GitHub and is visited by hundreds of thousands of developers every month.

Rank out of 28M!

359K

GitHub Stars

Star us on GitHub
Help us reach #1

+90k every month

+2.8M

Registered Users

Register yourself
Commit to your growth

+2k every month

49K

Discord Members

Join on Discord
Join the community

Roadmaps Guides FAQs YouTube

roadmap.sh by @nilbuild

Community created roadmaps, best practices, projects, articles, resources and journeys to help you choose your path and grow in your career.

THENEWSTACK

The top DevOps resource for Kubernetes, cloud-native computing, and large-scale development and deployment.

DevOps · Kubernetes · Cloud-Native

Cookie Settings